Challenge 6 — Fillable e Mass Assignment

Nov 9, 2026 · @Danilo Noccioli

Attacco

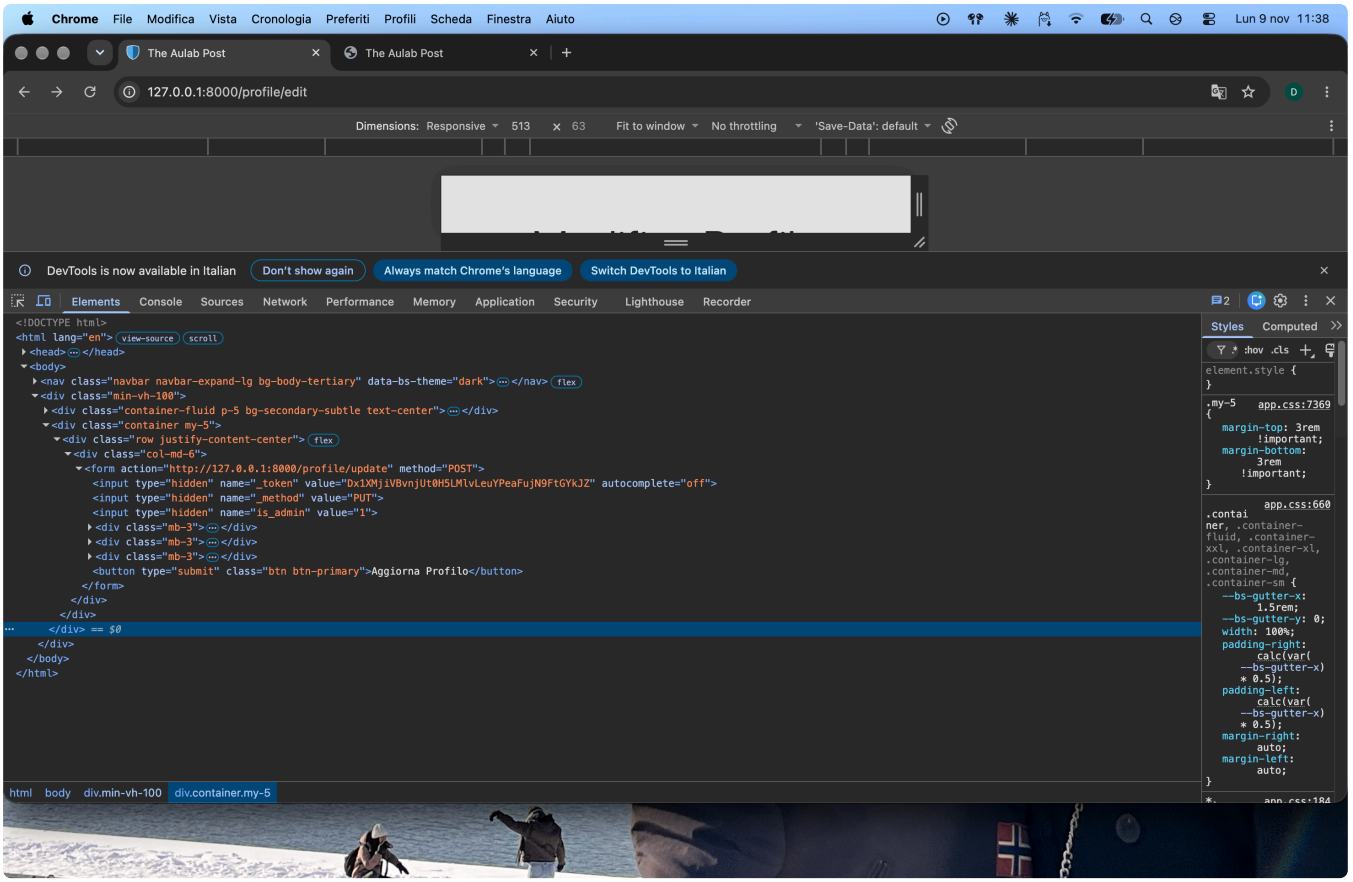
Setup vulnerabile

È stata creata una pagina di profilo utente (`ProfileController@edit` , `ProfileController@update` , vista `profile/edit.blade.php`) che permette di modificare nome, email e password. Nella versione vulnerabile il controller passava l'intera richiesta al modello:

Con `$fillable` che includeva ancora `is_admin` , qualunque campo extra nel form veniva accettato dal modello.

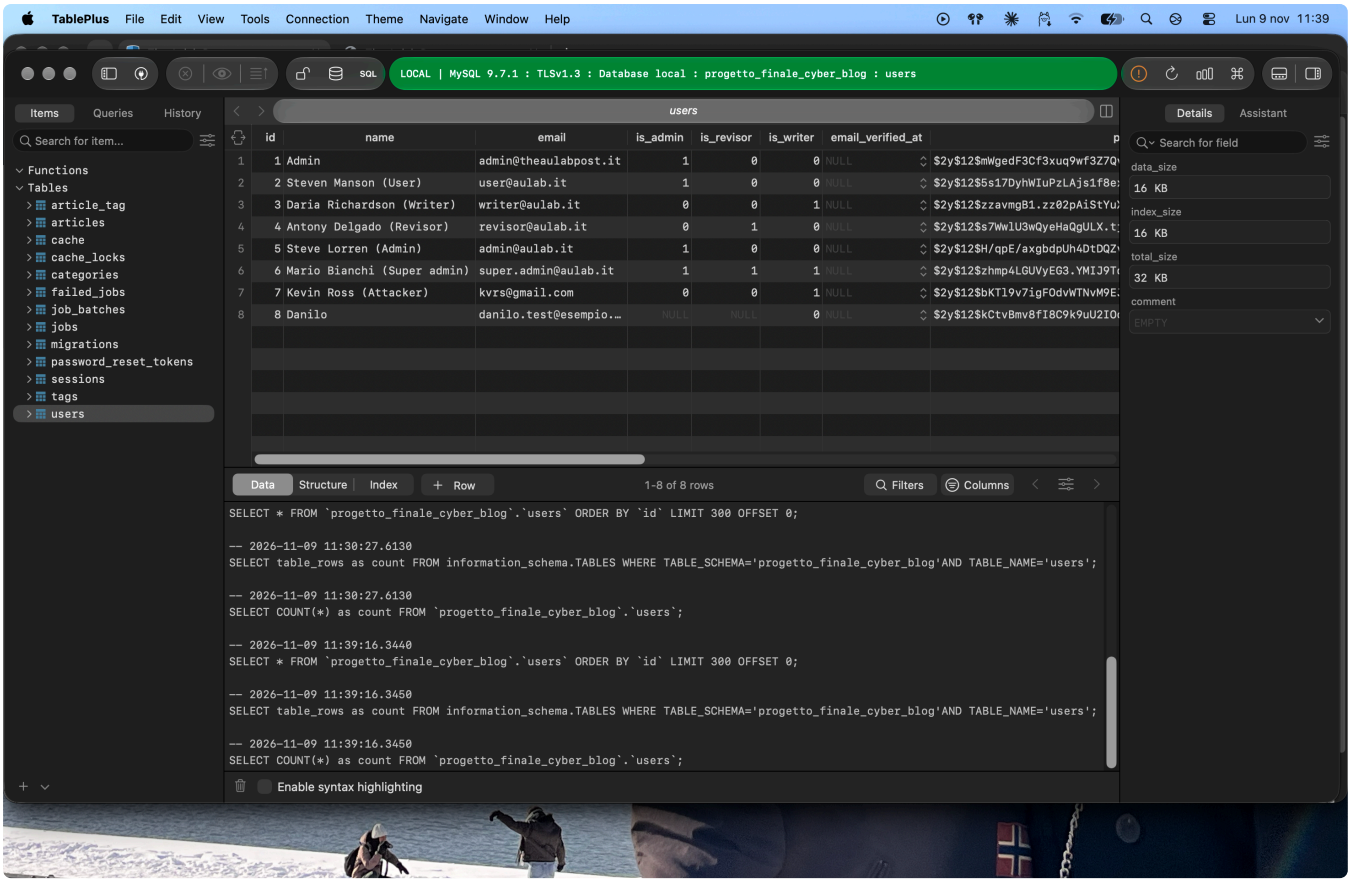
Esecuzione

1. Login con un utente normale (`user@aulab.it` , `is_admin = 0`).
2. Apertura di `/profile/edit` e ispezione del form.
3. Aggiunta, tramite "Edit as HTML" negli strumenti sviluppatore, di un input nascosto `<input type="hidden" name="is_admin" value="1">` dentro il form, insieme ai campi già presenti `_token` e `_method` .
4. Invio del form.



Risultato

Dopo l'invio, il campo `is_admin` dell'utente è passato da `0` a `1` nel database: l'utente si è auto-promosso ad amministratore, senza passare da nessuna funzione autorizzata.



Mitigazione

Tre interventi, coerenti tra loro.

1. Validazione esplicita nel controller

Al posto di `$request->all()`:

```
public function update(Request $request)
{
    $validatedData = $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email|max:255|unique:users,email,' . auth()-
>id(),
        'password' => 'nullable|string|min:8',
    ]);

    if (!$request->filled('password')) {
        unset($validatedData['password']);
    }

    $user = auth()->user();
    $user->update($validatedData);

    return redirect()->route('profile.edit')->with('success', 'Profilo
aggiornato.');
}
```

Solo i campi elencati in `validate()` possono comparire in `$validatedData`: qualunque altro campo inviato (`is_admin` incluso) viene scartato prima di arrivare al modello.

2. `$fillable` ristretto nel modello User

Seconda barriera, indipendente dal controller:

```
protected $fillable = [
    'name',
    'email',
    'password',
];
```

Così, anche se in futuro un altro controller passasse input grezzo al modello, i campi di ruolo resterebbero comunque protetti.

3. Adeguamento del seeder

`DatabaseSeeder` creava utenti con ruoli tramite `User::create([...'is_admin' => true, ...])`: mass assignment che si sarebbe rotto silenziosamente con `$fillable` ristretto. È stato corretto assegnando i ruoli per proprieta diretta dopo la creazione, lo stesso pattern già usato in `AdminController`:

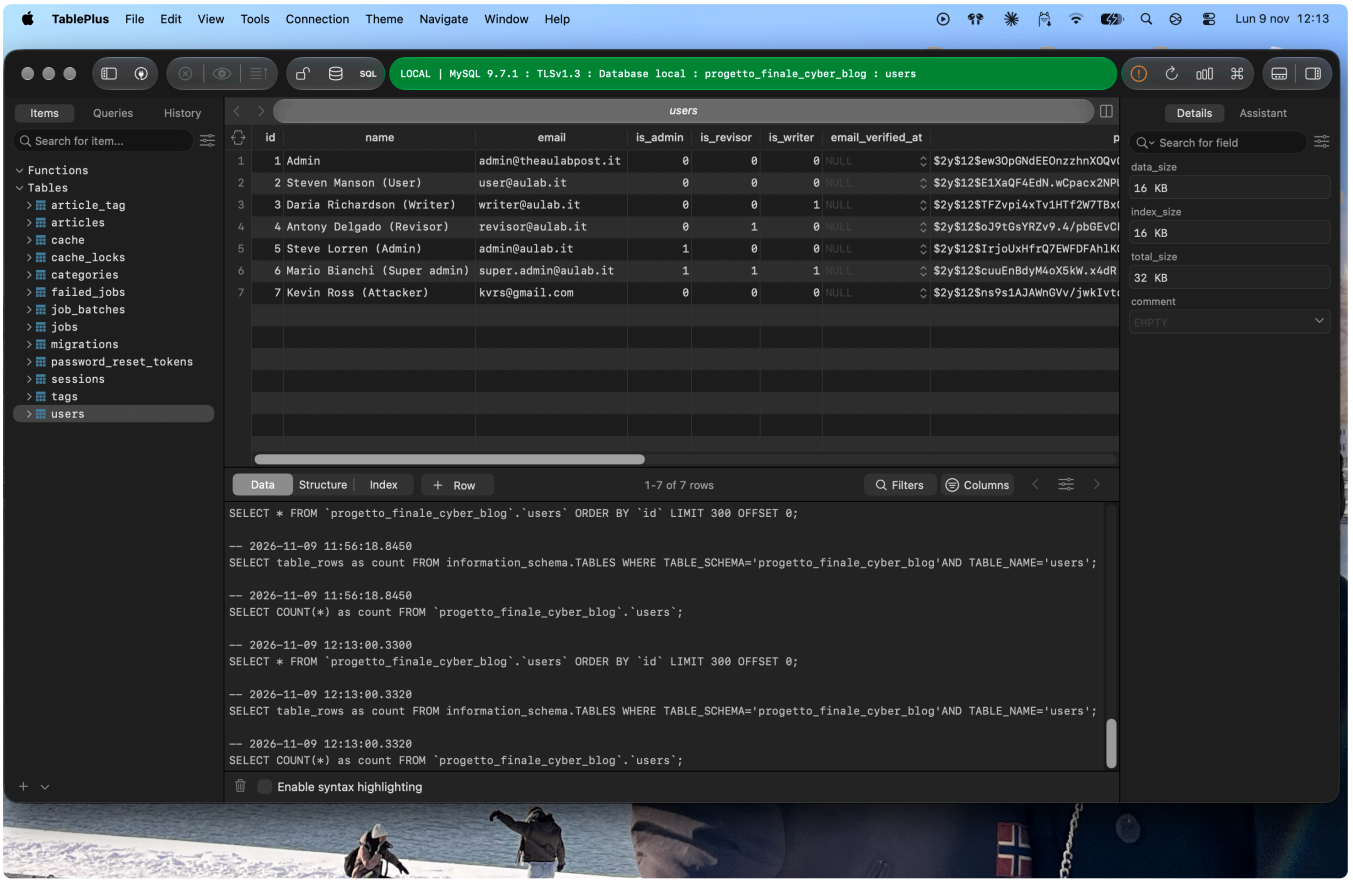
```
$user = User::create([
    'name' => "Steve Lorren (Admin)",
    'email' => 'admin@aulab.it',
    'password' => Hash::make('password'),
]);
$user->is_admin = true;
$user->is_writer = false;
$user->is_revisor = false;
$user->save();
```

4. Cambio ruolo isolato e autorizzato

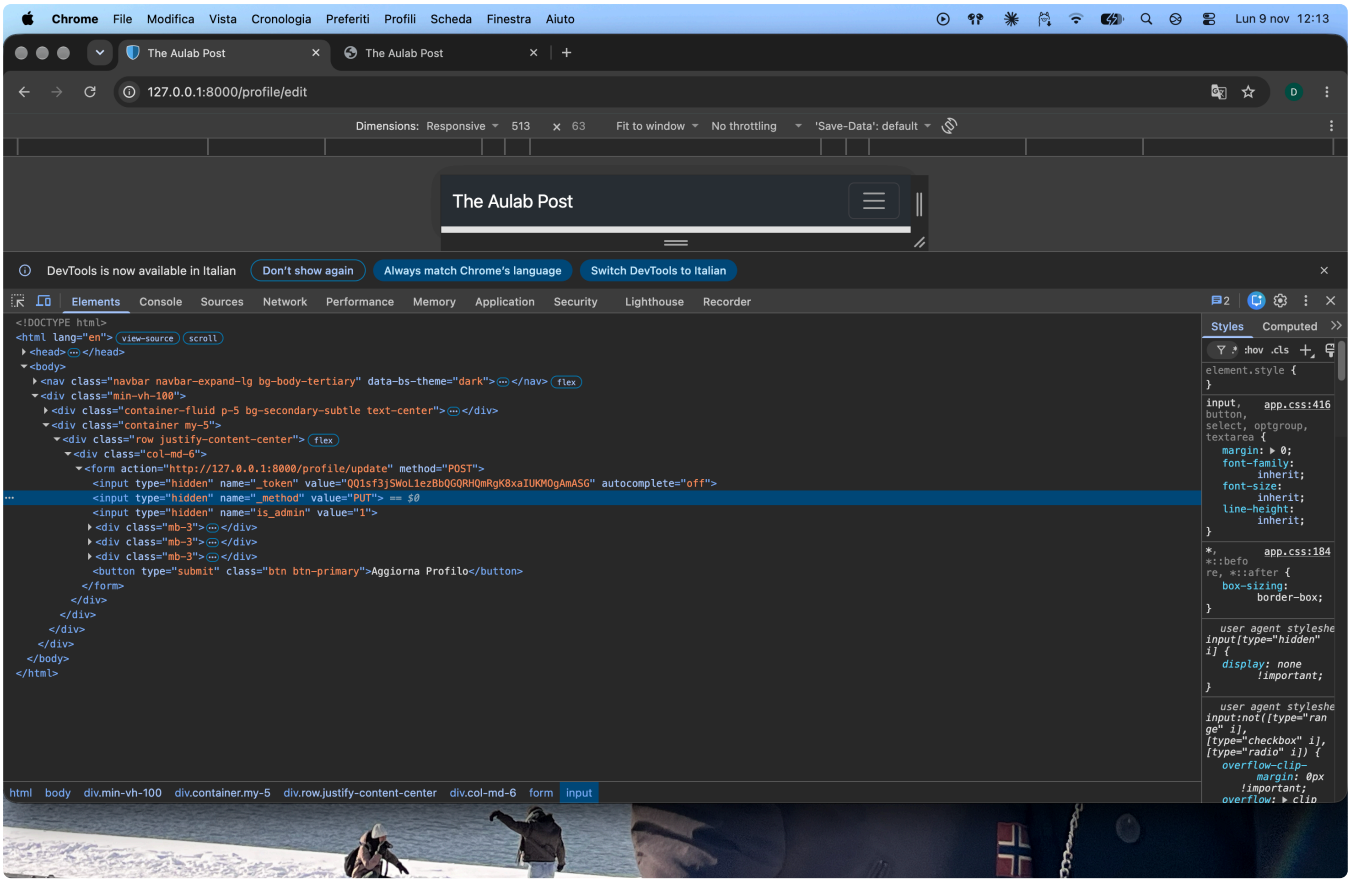
`AdminController@setAdmin/setRevisor/setWriter` imposta i ruoli per assegnazione diretta (non mass assignment) e logga l'operazione (attore, IP, valore prima/dopo). Le rotte corrispondenti sono protette da `middleware(['admin', 'admin.local'])`, quindi irraggiungibili da un utente non amministratore.

Verifica

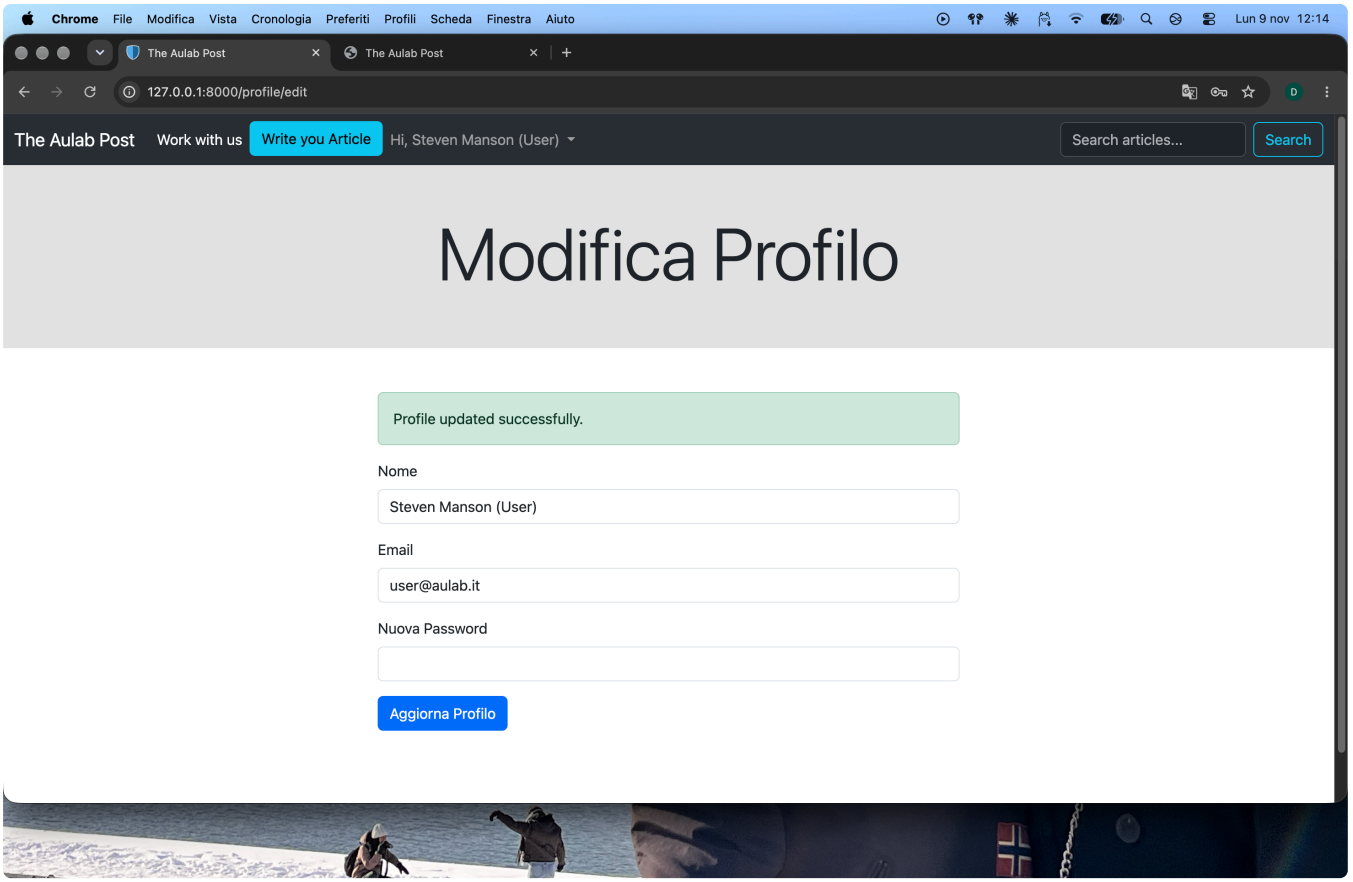
Dopo la mitigazione, il database è stato ripristinato (`php artisan migrate:fresh --seed`) con i ruoli corretti.



Lo stesso attacco è stato ripetuto: campo nascosto `is_admin=1` aggiunto al form via "Edit as HTML", form inviato con `_token`, `_method=PUT` e `is_admin=1`.



Il salvataggio è andato a buon fine ("Profilo aggiornato") ma il ruolo non è cambiato: l'utente è rimasto `user@aulab.it`, senza alcun ruolo, e il valore di `is_admin` nel database è rimasto `0`. Il campo extra è stato ignorato sia dalla validazione nel controller sia, a monte, da `$fillable`.


Chrome File Modifica Vista Cronologia Preferiti Profili Scheda Finestra Aiuto
Lun 9 nov 12:14
The Aulab Post
The Aulab Post
127.0.0.1:8000/profile/edit
The Aulab Post
Work with us
Write you Article
Hi, Steven Manson (User)
Search articles...
Search
Modifica Profilo
Profile updated successfully.
Nome
Steven Manson (User)
Email
user@aulab.it
Nuova Password
Aggiorna Profilo